

CSCI 357 Final Project Proposal

Course: CSCI 357 — AI and Neural Networks, Section 01

Semester: Spring 2026

Instructor: Prof. Brian King

Student: Nathan Casarsa

1. Project Title

MarketLens: An Explainable Neural Trading Signal Platform

2. What I Am Building

I am building an end-to-end neural market forecasting system that ingests multi-sector public equity data, utilizes various technical indicators with financial news sentiment, and produces explainable directional predictions via a live, interactive Streamlit dashboard deployed publicly on Hugging Face Spaces.

3. Neural Network Component

The core architecture is a **Temporal Fusion Transformer (TFT)**, a multi-horizon forecasting model designed to natively handle heterogeneous input types: static covariates (sector membership, market cap bucket), known future inputs (Fed meeting dates, earnings dates), and observed past inputs (OHLCV, 30+ technical indicators). The TFT is the right model here because vanilla Transformers are known to underperform on time series due to pointwise attention over raw timesteps; TFT addresses this with variable selection networks and interpretable multi-head attention over meaningful temporal patterns.

On top of the TFT, I plan to layer two additional neural components:

- **FinBERT sentiment encoder** — a pretrained BERT model fine-tuned on financial news, used to produce daily sentiment scores from RSS/NewsAPI headlines that are fed as an additional observed input to the TFT. This makes the system genuinely multimodal: price history, technical signals, and news sentiment all feed a single architecture.

- **LSTM autoencoder for regime detection** — a secondary unsupervised model that clusters market days into latent regime embeddings (e.g., trending, mean-reverting, high-volatility). Regime labels are surfaced in the dashboard and used to condition the interpretation of TFT predictions.

SHAP (specifically DeepSHAP) will be applied post-hoc to the TFT to produce per-prediction feature attribution, visualized as waterfall charts in the Streamlit app. Attention weights from the TFT's temporal self-attention will also be extracted and plotted as heatmaps over the past 30 trading days.

4. Data

- **Equity price and volume data:** [yfinance](#) (Yahoo Finance public API) — free, no key required, covers all major US equities with full OHLCV history. I will pull a universe of ~50 stocks across 5 sectors: tech, energy, financials, defense, and utilities.
- **Technical indicators:** computed locally using the [ta](#) Python library (RSI, MACD, Bollinger Bands, ATR, OBV, Stochastic, ADX, and ~25 others).
- **News sentiment:** NewsAPI (free tier) or Yahoo Finance RSS feeds for financial headlines, processed through FinBERT.
- **Event calendar:** Fed FOMC meeting dates and S&P 500 earnings dates sourced from public financial calendars (hardcoded or scraped once).

All data is publicly available. I have already verified [yfinance](#) access and pulled test data.

5. Tools and Stack

Component	Tool
Core model	PyTorch + PyTorch Forecasting (TFT implementation)
Sentiment model	HuggingFace Transformers (FinBERT: ProsusAI/finbert)
Technical indicators	ta library
Data ingestion	yfinance , pandas

Component	Tool
Explainability	shap (DeepSHAP), manual attention weight extraction
Experiment tracking	Weights & Biases (W&B sweeps for TFT hyperparameters)
Dashboard	Streamlit
Deployment	Hugging Face Spaces (free tier, public URL)
Development	Google Colab (GPU), local VS Code
Version control	GitHub

6. Stretch Goal

The most ambitious version of this project would wrap the trained TFT as an **MCP (Model Context Protocol) server**, allowing a Claude or other LLM agent to call the model as a tool at inference time. A user could ask a natural language question — "Is the model bullish on NVDA this week?" — and the agent would call the MCP endpoint, receive a structured prediction + SHAP attribution, and synthesize a grounded response. This would require implementing a lightweight MCP-compliant server around the PyTorch model inference pipeline. A secondary stretch goal is replacing the single TFT with a **Mixture of Experts** architecture, where one expert specializes per sector, with a learned router whose routing weights are logged and visualized in W&B.

7. Risk

The most likely bottleneck is the TFT training stability and data pipeline complexity. TFTs require careful alignment of static, known-future, and observed inputs, and PyTorch Forecasting's data format is more rigid than a standard DataLoader. A secondary risk is that FinBERT sentiment integration may be noisy enough to hurt rather than help model performance, which would require ablation experiments to diagnose. The SHAP computation over a Transformer is also compute-intensive; if DeepSHAP is too slow for the dashboard, I will fall back to a gradient-based saliency approach instead. I am planning to have a working TFT training loop and basic dashboard shell completed well before the April 27 checkpoint.